

HỒ SƠ THIẾT KẾ KIẾN TRÚC HỆ THỐNG

Bài tập 5 • Combo “Chuyến đi trọn gói”

Mô hình	Saga Orchestration
Phạm vi	Flight + Hotel + Payment
Công nghệ	Java 21 • Spring Boot • OpenFeign • Eureka • REST mock
Mục tiêu	Toàn bộ thành công hoặc tự động bù trừ

Tài liệu trình bày phân tích nghiệp vụ, lựa chọn Saga, kiến trúc module, cơ chế compensate, timeout/retry và bốn kịch bản phục hồi bắt buộc.

1. Bài toán và ràng buộc

Khách hàng đặt vé máy bay và phòng khách sạn trong cùng một yêu cầu. Combo chỉ được xác nhận khi Flight, Hotel và Payment đều thành công. Ba hệ thống sở hữu API và dữ liệu riêng, nên không thể dùng một transaction database chung hoặc dựa vào ACID xuyên service.

Giải pháp dùng chuỗi local transaction và compensating transaction để đạt eventual consistency. Đây là tính nguyên tử ở cấp nghiệp vụ, không phải rollback kỹ thuật tức thời.

Các dịch vụ tham gia

Dịch vụ	Trách nhiệm	Dữ liệu sở hữu
Combo Order	Nhận yêu cầu, điều phối, lưu state/trace	ComboOrder, sagald
Flight	Giữ và hủy vé máy bay	Flight reservation
Hotel	Giữ và hủy phòng	Hotel reservation
Payment	Thu và hoàn tiền	Payment transaction
Eureka	Service discovery (hạ tầng)	Service registry

Luồng giao dịch thuận

- Tạo ComboOrder PENDING và một sagald duy nhất.
- Gọi Flight giữ chỗ; thành công chuyển FLIGHT_RESERVED.
- Gọi Hotel giữ phòng; thành công chuyển HOTEL_RESERVED.
- Gọi Payment thu tiền; thành công chuyển PAYMENT_COMPLETED.
- Xác nhận ComboOrder COMPLETED khi mọi bước đã hoàn tất.

Điểm có thể thất bại

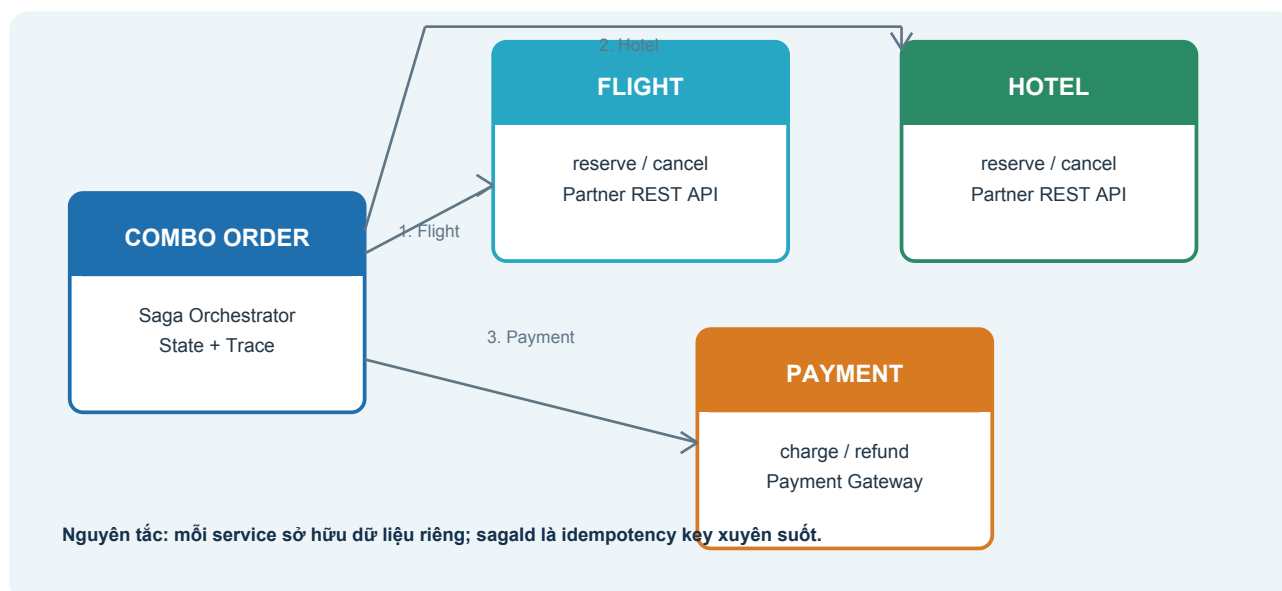
Điểm lỗi	Ví dụ	Cách xử lý
Flight	Hết vé, 5xx, timeout	Dừng Saga; chưa cần rollback bước trước
Hotel	Hết phòng, response chậm/mất	Cancel Hotel an toàn + cancel Flight
Payment	Bị từ chối hoặc gateway lỗi	Cancel Hotel rồi Flight
Compensate	Đối tác tạm unavailable	Retry bền vững, DLQ, cảnh báo
Orchestrator	Restart giữa Saga	Persist state + outbox trong production

2. Lựa chọn Saga Orchestration

Chọn Orchestration vì luồng combo có thứ tự rõ ràng, rollback phụ thuộc các bước đã hoàn thành và API của các đối tác không dùng chung một chuẩn sự kiện. Combo Order đóng vai trò điều phối trung tâm, giữ state machine và ra lệnh cho từng participant.

Tiêu chí	Orchestration (chọn)	Choreography
Luồng nghiệp vụ	Nhìn thấy tập trung, dễ kiểm soát	Phân tán qua nhiều event
Đối tác ngoài	Adapter REST phù hợp API khác nhau	Cần tích hợp event phức tạp
Compensate	Thứ tự ngược được xác định rõ	Dễ khó theo dõi quan hệ event
Quan sát/demo	Một sagald và trace xuyên suốt	Cần distributed tracing đầy đủ
Đánh đổi	Orchestrator cần HA/persistence	Coupling sự kiện và khó debug hơn

3. Kiến trúc module



Mỗi service chỉ cập nhật dữ liệu do mình sở hữu. Bản demo dùng ConcurrentHashMap; khi triển khai thực tế, mỗi service dùng database riêng và Combo Order persist Saga state.

4. Giao dịch thuận và cơ chế bù trừ

Bước thuận	Trạng thái	Compensate	Idempotency
Flight.reserve	FLIGHT_RESERVED	Flight.cancel	Trả booking cũ / cancel lặp an toàn
Hotel.reserve	HOTEL_RESERVED	Hotel.cancel	Tombstone chặn request đến trễ
Payment.charge	PAYMENT_COMPLETED	Payment.refund	Một payment theo sagald
Xác nhận combo	COMPLETED	CANCELLED nếu rollback	State theo sagald

Thứ tự compensate

Compensate chạy theo thứ tự ngược: **Payment** → **Hotel** → **Flight**. Với Payment bị từ chối, chưa có tiền bị capture nên chỉ hủy Hotel và Flight. Với Hotel lỗi/timeout, orchestrator vẫn gửi Hotel.cancel trước khi Flight.cancel.

Điểm quan trọng của timeout: không nhận được response không đồng nghĩa đối tác chưa xử lý. Lệnh cancel Hotel luôn được gửi với cùng sagald; tombstone cancelledSagas ngăn request cũ đến trễ tạo ghost booking.

State machine

Luồng thành công	PENDING → FLIGHT_RESERVED → HOTEL_RESERVED → PAYMENT_COMPLETED → COMPLETED
Luồng lỗi	Bất kỳ bước lỗi → COMPENSATING → CANCELLED
Bù trừ lỗi	COMPENSATING → COMPENSATION_FAILED → worker retry / DLQ

Nguyên tắc participant

- Mỗi command mang sagald làm idempotency key.
- Reserve/charge lặp lại trả kết quả cũ, không tạo giao dịch kép.
- Cancel/refund lặp lại vẫn trả trạng thái cuối hợp lệ.
- Không xóa dấu vết Saga; giữ audit log đủ lâu để reconciliation.

5. Luồng bốn kịch bản

Kịch bản 1 — Thành công toàn bộ

01	Combo	Tạo PENDING
02	Flight	reserve → RESERVED
03	Hotel	reserve → RESERVED
04	Payment	charge → CAPTURED
05	Combo	COMPLETED

Kịch bản 2 — Flight thành công, Hotel thất bại

01	Flight	reserve thành công
02	Hotel	reserve trả lỗi nghiệp vụ HOTEL_FAILURE
03	Compensate	Hotel.cancel idempotent → Flight.cancel
04	Combo	CANCELLED

Kịch bản 3 — Payment thất bại

01	Flight + Hotel	Cả hai reserve thành công
02	Payment	charge bị từ chối
03	Compensate	Hotel.cancel → Flight.cancel
04	Combo	CANCELLED

Kịch bản 4 — Hotel timeout

01	Flight	reserve thành công
02	Hotel	Mỗi response chậm 5 giây; client timeout 2 giây
03	Retry	Thử tối đa 3 lần với cùng sagald
04	Compensate	Hotel.cancel/tombstone → Flight.cancel
05	Combo	CANCELLED; request Hotel đến trễ không tạo booking

6. Timeout, retry và phục hồi

Chính sách trong bản demo

Tham số	Giá trị	Ý nghĩa
Connect timeout	1 giây	Không chờ lâu khi đối tác không kết nối được
Read timeout Hotel	2 giây	Giới hạn thời gian đợi response
Retry	Tối đa 3 lần	Cùng sagald, backoff 200 ms đến 1 giây
Mock timeout	5 giây	Buộc Feign phát sinh read timeout
Sau retry	Compensate	Hủy Hotel an toàn rồi hủy Flight

Quy tắc vận hành

- Chỉ retry lỗi tạm thời (connect/read timeout, 502/503/504); không retry hết phòng hoặc thanh toán bị từ chối.
- Dùng exponential backoff có jitter để tránh retry storm; deadline tổng phải nhỏ hơn timeout của client.
- Circuit breaker và bulkhead cô lập đối tác chậm, tránh làm cạn thread của Combo Order.
- Compensate thất bại phải được persist vào outbox, retry bởi worker và chuyển DLQ/cảnh báo khi quá ngưỡng.
- Reconciliation job đối chiếu định kỳ với đối tác để phát hiện booking/payment mờ côi.

Phân loại kết quả

Loại	Ví dụ	Quyết định
Thành công chắc chắn	HTTP 2xx + reservation ID	Đi tiếp
Thất bại chắc chắn	HTTP 409 hết phòng	Không retry; compensate
Không chắc chắn	Timeout / mất response	Retry idempotent rồi cancel an toàn
Bù trừ chưa xong	Cancel trả 503	COMPENSATION_FAILED + retry bền vững

7. API và cấu trúc source

Service	Port	API chính
Eureka	8761	Registry dashboard
Combo Order	8080	POST /api/v1/combo-orders; GET /{sagald}
Flight	8081	POST/DELETE /api/v1/flights/reservations
Hotel	8082	POST/DELETE /api/v1/hotels/reservations
Payment	8083	POST /charges; POST /refunds/{sagald}

Input mô phỏng

scenario	Kết quả mong đợi
SUCCESS	COMPLETED
HOTEL_FAILURE	CANCELLED; Flight đã rollback
PAYMENT_FAILURE	CANCELLED; Hotel và Flight đã rollback
HOTEL_TIMEOUT	CANCELLED sau retry; chống ghost booking

Cấu trúc nội bài

Mỗi service là một Gradle project độc lập, giống format project mẫu. File demo.http chứa bốn request; scripts/demo.sh chạy tuần tự; HO_SO_THIET_KE.md là bản hồ sơ có thể cập nhật; output/pdf chứa bản phát hành PDF.

Dữ liệu quan sát trong response

- sagald và trạng thái cuối của ComboOrder.
- flightReservationId, hotelReservationId, paymentId nếu bước tương ứng thành công.
- failureReason cho lỗi gốc.
- trace có timestamp cho từng transition và từng compensate.

8. Production readiness và video demo

Nâng cấp cần có khi triển khai thực tế

- Persist Saga state và optimistic locking; không phụ thuộc bộ nhớ tiến trình.
- Transactional outbox/inbox và broker cho command/compensate bền vững.
- Circuit breaker, bulkhead, rate limit và deadline propagation.
- mTLS/OAuth2 giữa service; mã hóa dữ liệu nhạy cảm và audit trail.
- Distributed tracing theo sagald; metric latency, retry, compensate và Saga bị treo.
- Reconciliation với Flight/Hotel/Payment và quy trình xử lý thủ công có kiểm soát.

Kịch bản quay video đề xuất

Bước	Nội dung trình bày
1	Giới thiệu sơ đồ module, Saga Orchestration và state machine
2	Mở Eureka, xác nhận bốn service nghiệp vụ đã đăng ký
3	Chạy SUCCESS, chỉ ra COMPLETED và ba ID
4	Chạy HOTEL_FAILURE, chỉ ra Hotel/Flight compensate
5	Chạy PAYMENT_FAILURE, chỉ ra thứ tự Hotel → Flight
6	Chạy HOTEL_TIMEOUT, giải thích retry và tombstone
7	Kết luận eventual consistency và giới hạn của demo in-memory

KẾT LUẬN — Thiết kế đáp ứng yêu cầu “hoặc thành công toàn bộ, hoặc hủy toàn bộ” ở cấp nghiệp vụ, có xử lý rõ kết quả không chắc chắn do timeout và có đường nâng cấp sang một Saga bền vững cho môi trường production.